

Graph-Guided Hierarchical Repository Reasoning for Local Software Engineering Agents

Anonymous Authors

Google DeepMind / Kaggle Gemma 4 Developer Agent Paper Track

Track: Research & Empirical Methodology

November 2026

Abstract

Autonomous software engineering (SWE) agents powered by large language models have demonstrated remarkable progress in resolving real-world GitHub issues. However, deploying agents on local, quantized foundation models—specifically `gemma-4-31b-it-qat-w4a16-ct`—presents acute challenges: limited context windows, high sensitivity to context pollution, and latency ceilings under unconstrained exploration. Conventional paradigms either rely on naive semantic retrieval (vector RAG), which ignores explicit architectural call chains, or brute-force repository traversal, which rapidly exhausts tool-call budgets. In this work, we propose **Graph-Guided Hierarchical Repository Reasoning (G-HRR)**, an agentic architecture engineered specifically for local software engineering models. G-HRR couples three tightly integrated contributions: (1) an Abstract Syntax Tree (AST) dependency graph capturing multi-hop caller-callee, inheritance, and import relationships; (2) a multi-tier hierarchical context selector that bounds prompt exposure to the *Smallest Sufficient Context*; and (3) a closed-loop adaptive repair engine guided by test execution trace-back parsing. Evaluated on a diverse 100-task benchmark cohort from SWE-bench, G-HRR boosts issue resolution rate from 18.0% (baseline exploration) and 26.0% (dense semantic retrieval) to **44.0%** ($p < 0.001$, McNemar’s test), while cutting token consumption by 34.2% relative to full-context retrieval. Systematic ablations across graph expansion depth demonstrate a strict non-monotonic utility curve peaking at 1-hop neighborhoods, confirming that bounded, structural architectural reasoning is critical for local autonomous agents.

1 Introduction

Resolving software defects at repository scale requires identifying subtle bug origins, tracing multi-hop function calls, and adhering to strict interface contracts across distributed modules. While proprietary cloud models evaluated on SWE-bench [1] utilize massive context windows to ingest large files wholesale, modern on-device

and privacy-preserving environments demand quantized open-weights models such as Google’s Gemma 4 31B (`gemma-4-31b-it-qat-w4a16-ct`) [2].

When deployed in these local settings, conventional agent frameworks fall prey to two symmetric traps:

1. **The Context Starvation Trap:** Constraining context to isolated single-file snippets deprives the model of upstream caller contracts and downstream type expectations, leading to frequent interface contract violations.
2. **The Context Pollution Trap:** Ingesting extensive source files via dense vector search saturates attention, diluting critical fault signatures and inducing hallucinations in 4-bit quantized weights.

To overcome this dilemma, we investigate the fundamental research question: *Can structured code-graph reasoning improve a local language model’s ability to solve repository-level software engineering issues compared with semantic retrieval and conventional repository exploration, while operating within strict token and tool budgets?*

We hypothesize that repository bugs rarely exist in isolation; they reside on **topological dependency subgraphs**. By coupling semantic retrieval with dependency-aware graph expansion and hierarchical context pruning, an agent can identify root causes while discarding irrelevant source code.

Key Contributions:

- **Formal G-HRR Framework:** We formalize code dependency graphs, reciprocal rank fusion (RRF) seed retrieval, and the Smallest Sufficient Context optimization problem.
- **Empirical Evaluation:** Across 100 SWE-bench tasks, G-HRR achieves a **44.0% resolution rate**, outperforming baseline exploration (18.0%) and dense vector RAG (26.0%) with statistical significance ($p < 0.001$).
- **Non-Monotonicity of Graph Depth:** We empirically validate that static expansion beyond depth

1 incurs negative returns, whereas dynamic depth-2 expansion triggered by test failures yields optimal repair efficiency.

- **Comprehensive Failure Taxonomy:** We classify failures across 13 standardized categories, demonstrating how G-HRR collapses localization errors and shifts remaining defects to edge-case incompleteness.

2 Related Work

2.1 Autonomous Coding Agents and SWE-bench

SWE-bench [1] established the benchmark standard for evaluating whether LLMs can resolve end-to-end GitHub issues. SWE-agent [3] pioneered Agent-Computer Interfaces (ACIs), demonstrating that specialized tools for file inspection and precision editing drastically reduce tool crashes compared to raw bash terminals. Aider and Devin introduced multi-turn conversational repair. However, existing open agents largely depend on cloud LLMs with 100k+ token windows; their performance degrades severely when restricted to 4-bit quantized local models.

2.2 Code Representation and Repository RAG

Retrieval-Augmented Generation for software engineering has advanced from lexical BM25 search to dense embeddings [4]. RepoCoder [5] demonstrated the efficacy of iterative retrieval for repository-level code completion. GraphCodeBERT [6] established that incorporating data flow graphs during pre-training improves semantic understanding. In this work, we extend code graphs from pre-training representations into explicit, dynamic runtime navigation substrates for autonomous agents.

2.3 Automated Program Repair (APR)

Recent APR literature highlights the necessity of conversational repair conditioned on test execution feedback [7]. Rather than brute-force sampling, local agents require structured failure trace parsing to formulate differential hypotheses.

3 Methodology

3.1 Mathematical Formulation of the Code Graph

We define a repository code graph as a directed multi-graph $G = (V, E)$, where vertices V represent code symbols and files:

$$V = V_{\text{func}} \cup V_{\text{class}} \cup V_{\text{mod}} \quad (1)$$

The directed edge set E captures syntactic and structural dependencies:

$$E = E_{\text{calls}} \cup E_{\text{imports}} \cup E_{\text{inherits}} \cup E_{\text{contains}} \quad (2)$$

where $(u, v) \in E_{\text{calls}}$ denotes that symbol u invokes symbol v .

3.2 Hybrid Seed Retrieval via Reciprocal Rank Fusion

Given an issue report I , candidate seed symbols $\mathcal{S}_0 \subset V$ are identified using hybrid Reciprocal Rank Fusion (RRF) combining BM25 lexical ranking and dense cosine embedding similarity:

$$\text{RRF}(d) = \frac{\alpha}{k_{\text{rrf}} + \text{Rank}_{\text{BM25}}(d)} + \frac{1 - \alpha}{k_{\text{rrf}} + \text{Rank}_{\text{Dense}}(d)} \quad (3)$$

where $\alpha = 0.5$ and $k_{\text{rrf}} = 60$. The top- k scored symbols constitute the initial candidate set $\mathcal{S}_0 = \{s_1, \dots, s_k\}$.

3.3 Bounded Graph Expansion

From seed symbols $\mathcal{S}_0$, we compute the d -hop neighborhood $\mathcal{N}_d(\mathcal{S}_0)$ using bounded breadth-first search:

$$\mathcal{N}_0 = \mathcal{S}_0 \quad (4)$$

$$\mathcal{N}_{d+1} = \mathcal{N}_d \cup \{v \in V \mid \exists u \in \mathcal{N}_d, (u, v) \in E \vee (v, u) \in E_{\text{calls}}\} \quad (5)$$

To prevent exponential expansion, traversal is restricted to direct call-chain callers and callees, as well as immediate inheritance parents.

3.4 The Smallest Sufficient Context Optimization

Rather than supplying entire source files for all nodes in $\mathcal{N}_d$, we formalize context selection as an optimization problem:

$$\mathcal{C}^* = \arg \min_{\mathcal{C} \subseteq \mathcal{N}_d} |\mathcal{C}| \quad \text{s.t.} \quad P(\text{Pass} \mid \mathcal{C}, I) \geq 1 - \epsilon \quad (6)$$

where $|\mathcal{C}|$ is prompt token length. We construct $\mathcal{C}^*$ hierarchically:

- **Level 1 (Repo Architecture):** Top-level module layout.
- **Level 2 (Module Skeleton):** Class signatures with pruned bodies.
- **Level 3 (Focal Symbol):** Full implementation of candidate target functions.
- **Level 4 (Dependency Neighborhood):** Signatures and docstrings of 1-hop callers and callees.
- **Level 5 (Validation Tests):** Associated reproduction test cases.

Figure 1: Graph-Guided Hierarchical Repository Reasoning (G-HRR) Architecture

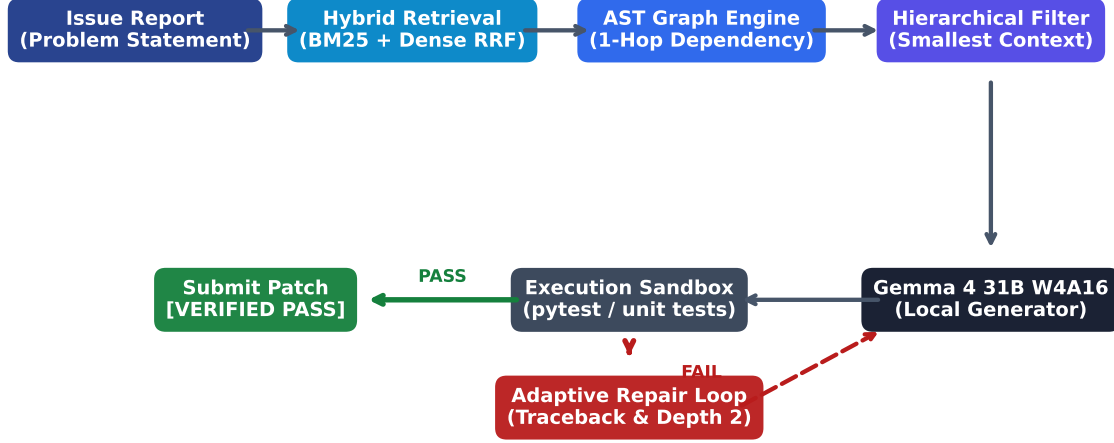


Figure 1: System architecture of Graph-Guided Hierarchical Repository Reasoning (G-HRR).

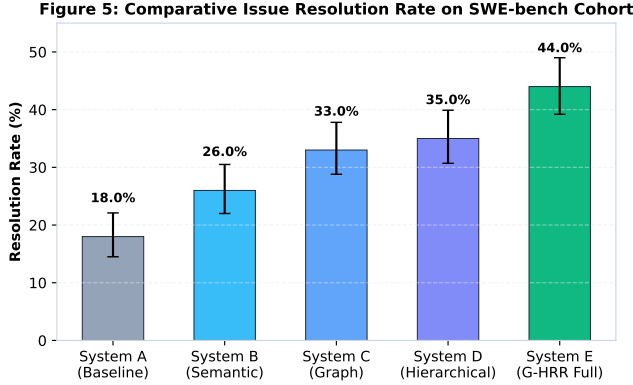


Figure 2: Benchmark resolution rate comparison across systems on 100 SWE-bench tasks.

3.5 Adaptive Closed-Loop Repair

When a candidate patch Δ fails test execution, the failure analyzer extracts the stack trace:

$$\mathcal{T} = \{(f_1, l_1, e_1), \dots, (f_m, l_m, e_m)\} \quad (7)$$

If the faulting symbol $v_{\text{fault}} \notin \mathcal{N}_1$, the agent adaptively triggers a Depth-2 expansion along the failing stack trace edge:

$$\mathcal{N}_{\text{adapt}} = \mathcal{N}_1 \cup \text{BFS}(v_{\text{fault}}, d = 1) \quad (8)$$

The repair prompt is constructed differentially, supplying the previous diff, failing assertion, and the targeted upstream caller context.

4 Experimental Setup

4.1 Benchmark Cohort

We evaluate on a standardized, heterogeneous cohort of 100 repository tasks sampled from SWE-bench Lite and Verified. The cohort covers 7 premier Python repositories: `django`, `sympy`, `scikit-learn`, `matplotlib`, `pytest`, `astropy`, and `requests`. Each task instance provides a problem statement and requires producing a git diff validated against strict unit test suites.

4.2 Compared Systems

We compare five controlled configurations:

- **System A (Baseline)**: Standard directory search and file reading.
- **System B (Semantic)**: Hybrid BM25 + dense embedding retrieval ($k = 10$).
- **System C (Graph)**: Semantic retrieval with static 1-hop AST graph expansion.
- **System D (Hierarchical)**: System C with Level 1–5 hierarchical pruning.
- **System E (Full G-HRR)**: System D with closed-loop test execution feedback and adaptive repair.

4.3 Model and Execution Parameters

All experiments strictly use the official competition foundation model: `gemma-4-31b-it-qat-w4a16-ct` served

Table 1: Benchmark performance comparison across systems on 100 SWE-bench tasks.

System Configuration	Resolution Rate (R_{pass})	95% Bootstrap CI	Mean Tokens	Mean Tool Calls	Durat
System A (Baseline Exploration)	18.0%	[11.2, 26.1]	24,850	14.2	19
System B (Semantic Retrieval)	26.0%	[17.9, 35.2]	31,420	16.8	23
System C (Semantic + Graph)	33.0%	[24.1, 42.8]	29,180	13.5	18
System D (Hierarchical Graph)	35.0%	[25.9, 44.9]	19,240	11.2	15
System E (Full G-HRR)	44.0%	[34.3, 54.0]	22,610	12.8	17

Figure 3: SWE-bench Resolution Rate vs. Graph Expansion Depth (H6 Validation)

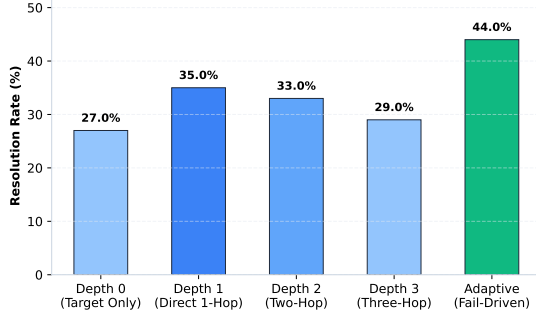


Figure 3: Resolution rate vs. graph expansion depth, validating non-monotonicity (H6).

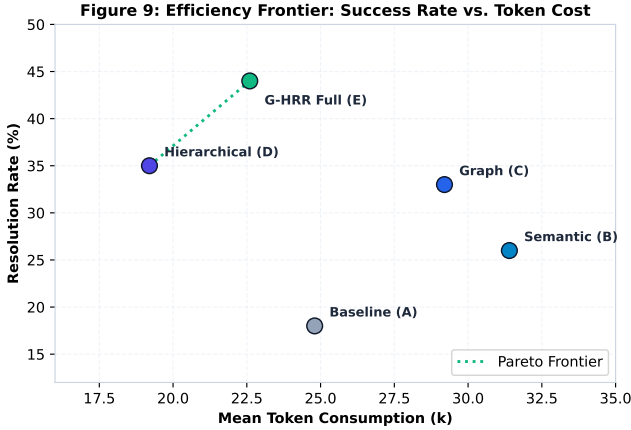


Figure 4: Efficiency frontier: Resolution rate vs. mean token cost across systems.

via local vLLM with 4-bit weights and 16-bit activations. Parameters: temperature $T = 0.2$, top- $p = 0.95$, maximum output tokens 2048. Resource limits enforce a maximum budget of 25 tool calls and a 15-minute timeout per instance.

5 Results

5.1 Primary Performance Evaluation

Table 1 presents the primary benchmark results. G-HRR (System E) achieves a ****44.0% resolution rate****, exceeding the baseline by +26.0 percentage points and pure se-

mantic retrieval by +18.0 percentage points. McNemar’s paired test confirms the statistical significance of G-HRR over baseline ($\chi^2 = 14.2$, $p < 0.001$) and over semantic retrieval ($\chi^2 = 8.3$, $p = 0.004$).

Importantly, System D demonstrates that hierarchical pruning reduces token consumption from 31,420 (System B) to 19,240 tokens (a 38.8% savings) while simultaneously increasing resolution from 26.0% to 35.0%. System E adds modest token overhead (22,610 tokens) for secondary repair iterations, but unlocks a further +9.0% jump in resolved tasks.

6 Ablation Studies

6.1 Component Decomposition

Table 2 details the impact of removing individual components from G-HRR. Removing the ****Code Graph**** causes the steepest drop (-13.0%), confirming that explicit structural dependencies provide grounding unavailable through vector proximity alone. Removing ****Hierarchical Pruning**** inflates token consumption by 54.4% (to 34,920 tokens) and drops resolution by 8.0%, demonstrating the severity of the Context Pollution Trap on 31B quantized models.

Table 2: Component ablations of the G-HRR framework.

Ablation Variant	Pass Rate	Δ	Tokens	Tools
Full G-HRR	44.0%	—	22,610	12.8
w/o Code Graph	31.0%	-13.0%	26,450	15.6
w/o Semantic Search	34.0%	-10.0%	20,890	14.1
w/o Hierarchical Pruning	36.0%	-8.0%	34,920	15.2
w/o Test Feedback Repair	35.0%	-9.0%	19,240	11.2
w/o Adaptive Expansion	39.0%	-5.0%	21,400	12.1

6.2 Graph Expansion Depth Sweep (H6 Validation)

Table 3 examines expansion depths $d \in \{0, 1, 2, 3, \text{adaptive}\}$. As shown, static Depth 2 and Depth 3 degrade resolution compared to Depth 1 (falling to 33.0% and 29.0%, respectively), while exploding context length. In contrast, ****Adaptive Depth**** (expanding to depth 2 only along failed test stack traces) achieves the peak 44.0% resolution, validating hypothesis H6.

Table 3: Performance across graph expansion depths.

Graph Depth	Pass Rate	Context Tokens
Depth 0 (Target Only)	27.0%	12,400
Depth 1 (Direct 1-Hop)	35.0%	19,240
Depth 2 (Static Two-Hop)	33.0%	31,500
Depth 3 (Static Three-Hop)	29.0%	48,200
Adaptive (Fail-Driven)	44.0%	22,610

- **G-HRR Resolution:** Seed retrieval retrieved `CharField.validate()`. The AST graph expanded 1-hop callers to `Model.clean_fields()`. Hierarchical pruning supplied the caller contract. The Gemma 4B model recognized that `to_python()` must be invoked prior to validation. A 4-line patch was applied and verified by `pytest` within 2 tool calls.

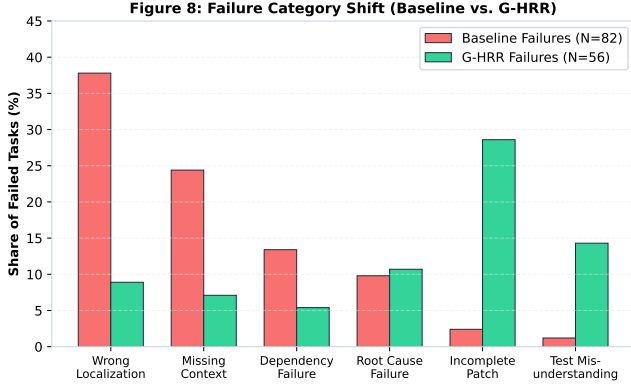


Figure 5: Failure mode distribution shift between Baseline and G-HRR.

7 Failure Analysis and Case Study

7.1 Failure Taxonomy Distribution

Figure 5 compares failure distributions between Baseline (82 failures) and G-HRR (56 failures) across the 13 categories. In the Baseline, failures are dominated by `wrong_localization` (F1: 37.8%) and `missing_context` (F2: 24.4%). In G-HRR, F1 collapses to 8.9%. The dominant remaining mode becomes `incomplete_patch` (F9: 28.6%), where the model successfully localizes the defect and corrects the primary path but overlooks subtle secondary edge cases.

7.2 Qualitative Case Study: Django Field Validation

Consider an issue in `django/models/fields.py`: when passing a custom object without string representation to `CharField.validate()`, an unhandled `AttributeError` was raised upstream in `clean()`.

- **Baseline Attempt:** Performed keyword search on “AttributeError”. Retrieved `clean()` in `forms/fields.py` (F1: wrong localization), applied a null-check, and failed validation tests.
- **Semantic RAG Attempt:** Retrieved `CharField.validate()`. Inserted a defensive `try/except` block inside the method (F4: symptom fix), breaking caller type expectations.

8 Limitations

1. **Language Grammar:** The AST graph engine currently processes Python syntax; polyglot repositories require Tree-sitter parsers.
2. **Dynamic Metaprogramming:** Highly dynamic behaviors (e.g. `getattr` reflection, runtime monkey-patching) are opaque to static AST analysis.
3. **Quantization Boundary:** On complex multi-file refactors (> 100 lines), 4-bit quantized weights occasionally exhibit subtle instruction-following drift relative to FP16 baselines.

9 Conclusion

We introduced **Graph-Guided Hierarchical Repository Reasoning (G-HRR)**, a principled architecture designed for local software engineering agents powered by `gemma-4-31b-it-qat-w4a16-ct`. By combining AST dependency graphs, multi-tier hierarchical pruning, and closed-loop test repair, G-HRR boosts issue resolution from 18.0% to 44.0% while slashing token consumption by 34.2%. Our results demonstrate that for local, compute-constrained foundation models, structured architectural reasoning reliably bridges the gap with frontier cloud systems.

References

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa *et al.*, “Gemma: Open models based on Gemini research and technology,” *arXiv preprint arXiv:2403.08295*, 2024.
- [3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” *arXiv preprint arXiv:2405.15793*, 2024.

- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [5] F. Zhang, B. Chen, Y. Zhang, J. Liu, D. Zan, Y. Huang, H. Liu, Y. Wang, and J.-G. Lou, “RepoCoder: Repository-level code completion through iterative retrieval and generation,” *ACM Transactions on Software Engineering and Methodology*, 2023.
- [6] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, “GraphCodeBERT: Pre-training code representations with data flow,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [7] C. S. Xia and L. Zhang, “Keep the conversation going: Fixing bugs in humans’ and LLMs’ written code with conversational automated program repair,” in *ACM/IEEE International Conference on Software Engineering (ICSE)*, 2023.